

Hybrid Computing Technology

1

Hybrid Computing Programming Architecture

ผศ. อัครินทร์ คุณกิตติ

ASST. PROF. AKHARIN KHUNKITTI

FACULTY OF INFORMATION TECHNOLOGY
KMITL



Topics

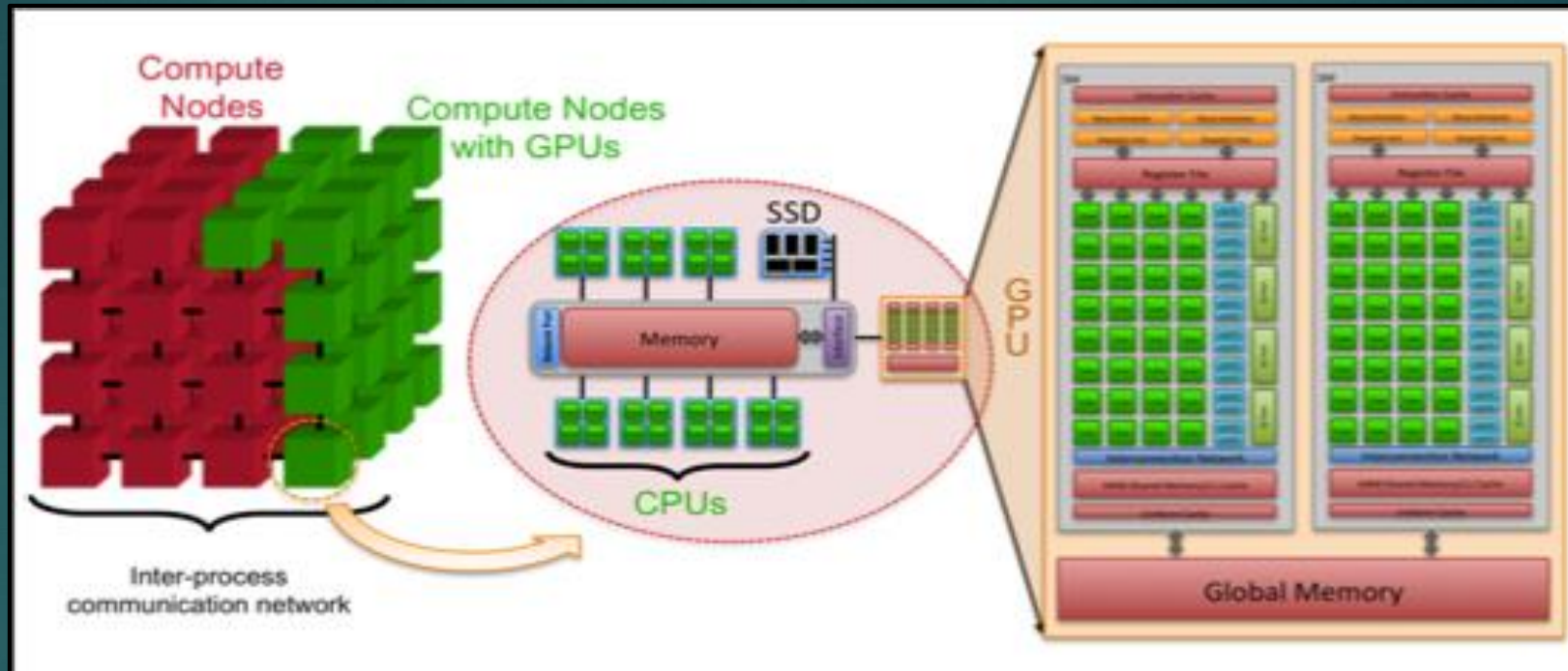
2

- ▶ Introduction to Hybrid Computing Programming
- ▶ Programming Architecture in Accelerating Computing
- ▶ Conclusion



Heterogeneous High-Performance System

3



Each node has multiple CPU cores, and some of the nodes are equipped with additional computational accelerators, such as GPUs.

Programming Challenges

- ▶ Scale and structure of parallelism
 - ▶ Hierarchical parallelism
 - ▶ Many more cores, more nodes in clusters
 - ▶ Heterogeneous nodes
- ▶ New methods and models to match architecture
 - ▶ Exploit intranode concurrency, heterogeneity
 - ▶ Adapt to reduced memory size; drastically reduce data motion
- ▶ Resilience in algorithm; fault tolerance in application
- ▶ Need to run multi-executable jobs
 - ▶ Coupled computations; Postprocessing of data while in memory
- ▶ Uncertainty quantification to clarify applicability of results

Programming Models: Today's Scenario

5

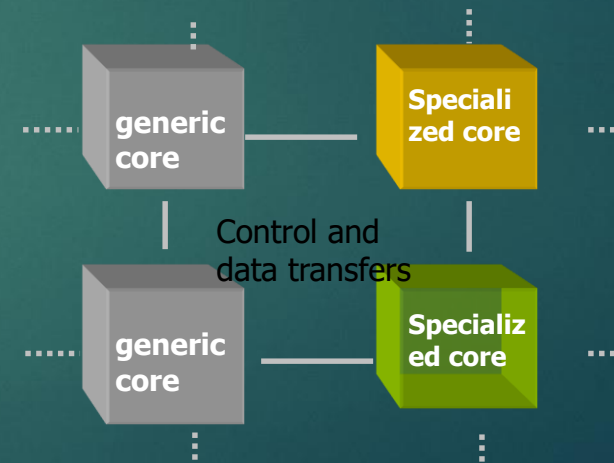
```
// Run one OpenMP thread per device per MPI node
#pragma omp parallel num_threads(devCount) if (initDevice())
{
    // Block and grid dimensions
    dim3 dimBlock(12,12);
    kernel<<<1,dimBlock>>>();
    cudaThreadExit();
}
else
{
    printf("Device error on %s\n",processor_name);
}
MPI_Finalize();
return 0;
}
```



Programming Model Major Requirements

6

- ▶ Portable expression of scalable parallelism
 - ▶ Across exascale platforms and intermediate systems
- ▶ Uniformity
 - ▶ One model across variety of resources
 - ▶ Across node, across machine?
- ▶ Locality
 - ▶ For performance and power
 - ▶ At all levels of hierarchy
- ▶ Asynchrony
 - ▶ Minimize delays
 - ▶ But trade-off with locality



Runtime Is Critical for Performance

7

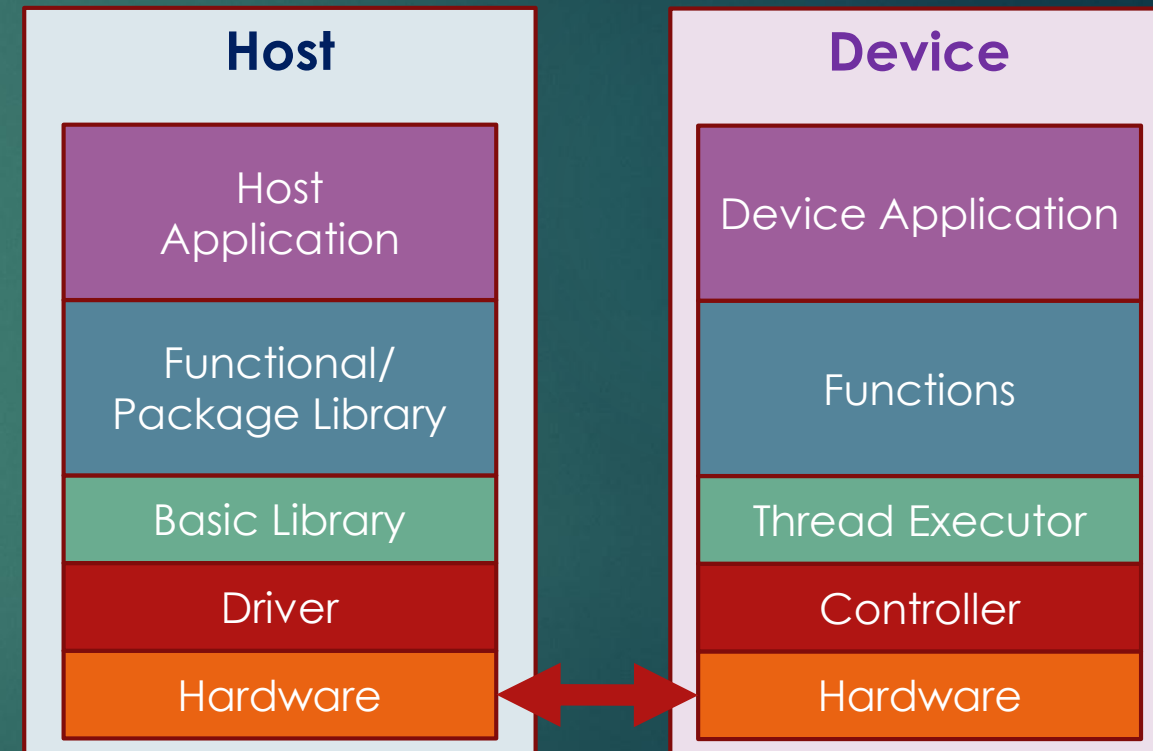
- ▶ Runtime support must
 - ▶ Adapt workload and data to environment
 - ▶ Respond to changes caused by application characteristics, power, faults, system noise
 - ▶ Dynamically and continuously
- ▶ Provide feedback on application behavior
- ▶ Uniform support for multiple programming models on heterogeneous platforms
 - ▶ Facilitate interoperability, (dynamic) mapping

Programming Architecture in Accelerating Computing

Physical Architecture

9

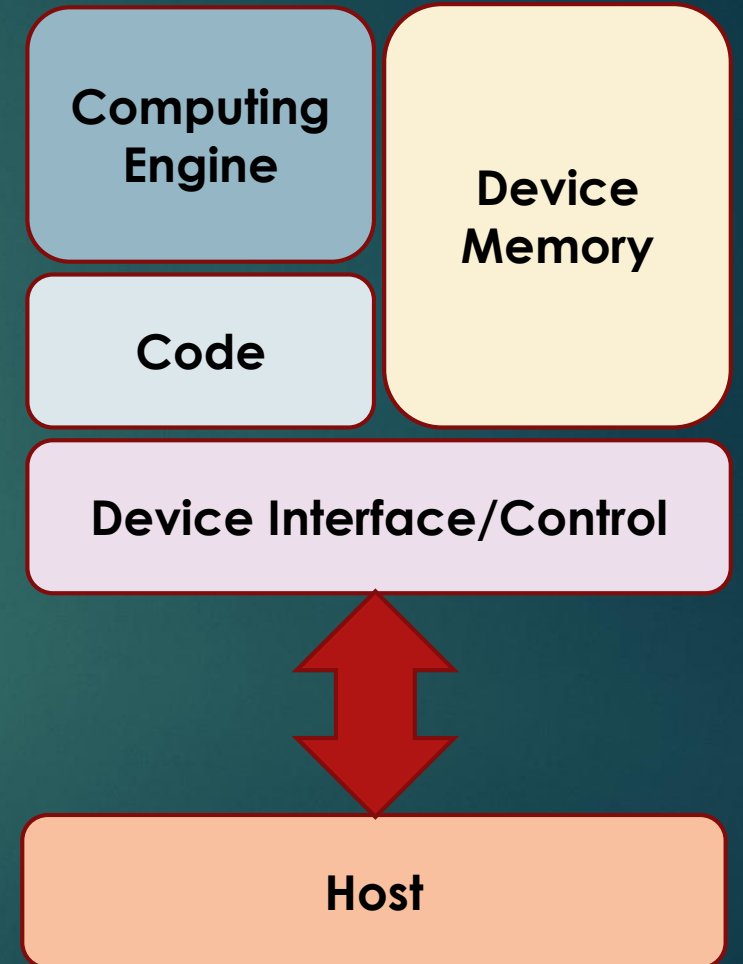
- ▶ Host-Devices Model
- ▶ Device
 - ▶ Computing Engine
 - ▶ Instruction-Based Logics
 - ▶ Direct Logics
 - ▶ Device Memory - DRAM
 - ▶ Interfacing – PCIe => Data Transfer + Control
- ▶ Host
 - ▶ Interfacing – PCIe => Data Transfer + Control
 - ▶ Host Memory
 - ▶ CPU/Processors



Programming Architecture

10

- ▶ Device Programming
 - ▶ Code Location
 - ▶ Data Transfer => From/To Host Memory
 - ▶ Code Execution => Threads
 - ▶ Device Memory Access for Processing/Computing
 - ▶ Read
 - ▶ Write
- ▶ Data Processing/Computing
 - ▶ Multiple Parallel Threads => Number of Threads
 - ▶ Pipelining

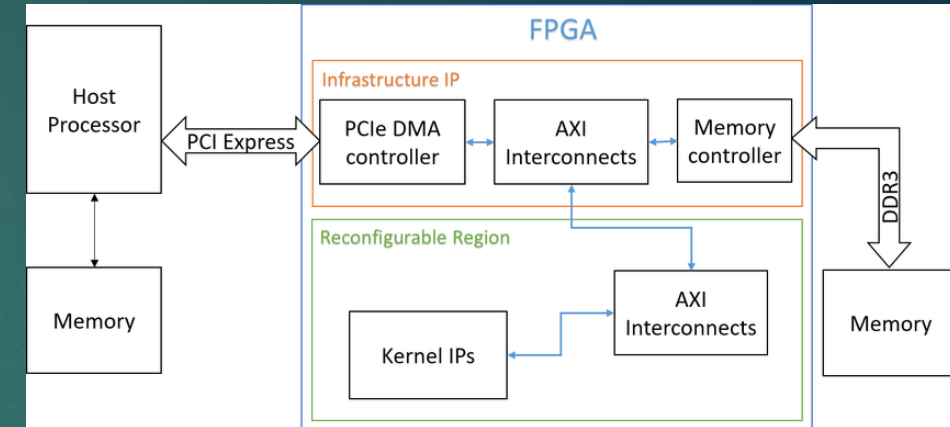


Programming Architecture

11

► Host Programming

- Initial Devices
- Code Loading
- Data Transfer From Host to Device Memory
- Thread Creation
 - Number of Threads
- Execution
 - Limitations
- Data Transfer From Device to Host Memory
- Code Un-Loading (Option)



The Cost of Memory Access

12

- ▶ Registers are extremely fast, but are a limited resource
- ▶ Cached memories also tend to be small
- ▶ For large data sets, **global memory provides read & write access**
 - ▶ Accesses take between 600 and 800 clock cycles
 - ▶ **Accesses are *not* cached**
 - ▶ To hide memory latency, the hardware provides **fast context switches when memory is accessed**
 - ▶ However, there must be enough computational work to do to hide the high cost of memory access
- ▶ Programmers need to be smart
 - ▶ Compilers often don't provide the necessary optimizations when optimizing for speed instead of code size
 - ▶ It can sometimes be **cheaper to recompute a result** than perform a memory read/write

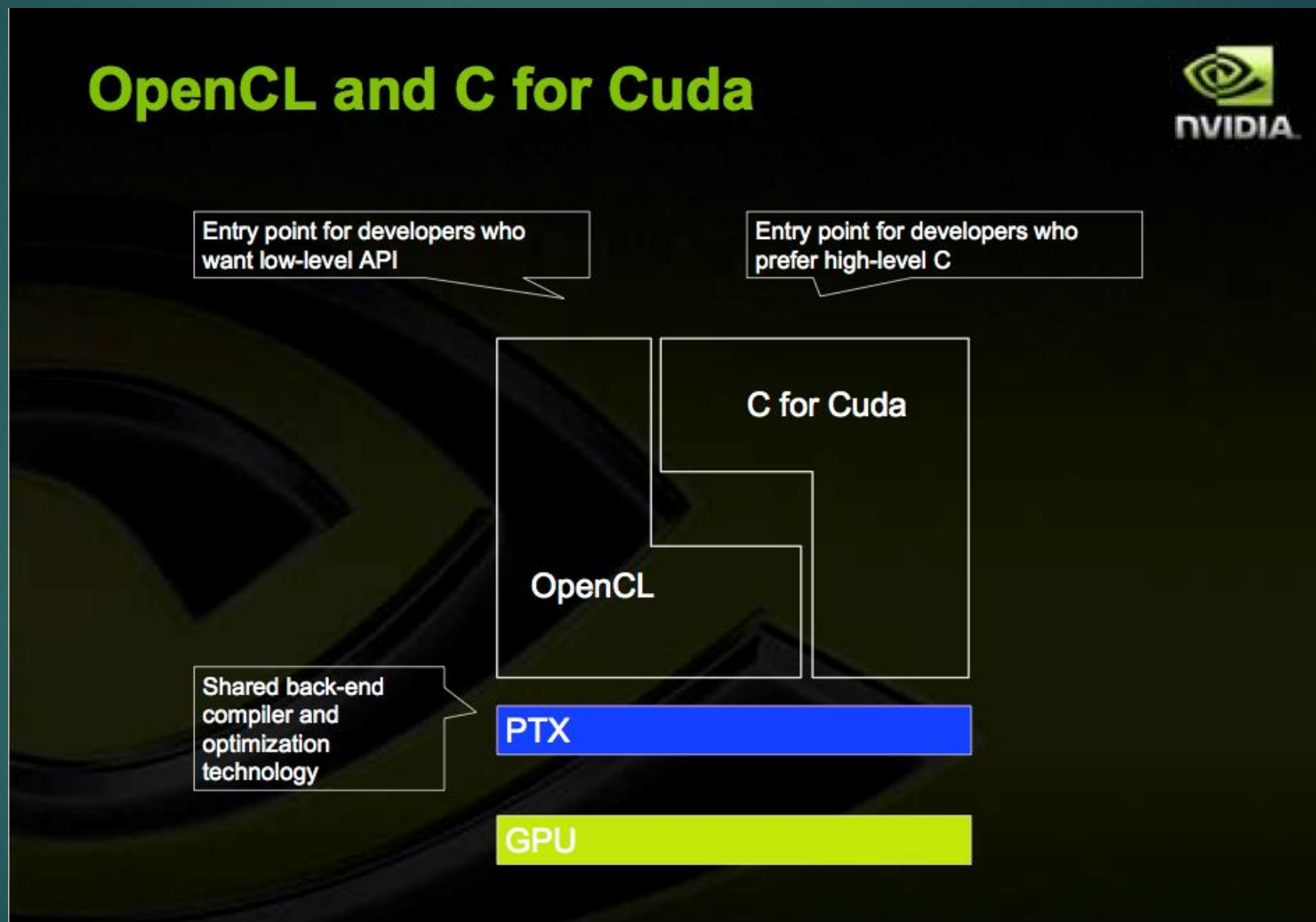
OpenCL vs CUDA

13

- ▶ CUDA uses **early code binding**
 - ▶ Code is compiled with normal C/C++/FORTRAN (beta) source code
 - ▶ Need CUDA **occupancy calculator** to determine number of threads based on resource utilization
 - ▶ Library support: **BLAS & FFT & DPT**
- ▶ OpenCL
 - ▶ **Late binding of OpenCL** code to executable
 - ▶ OpenCL compiler/linker embedded within application
 - ▶ No need for CUDA occupancy calculator
 - ▶ Only supports C
 - ▶ No libraries

OpenCL vs CUDA

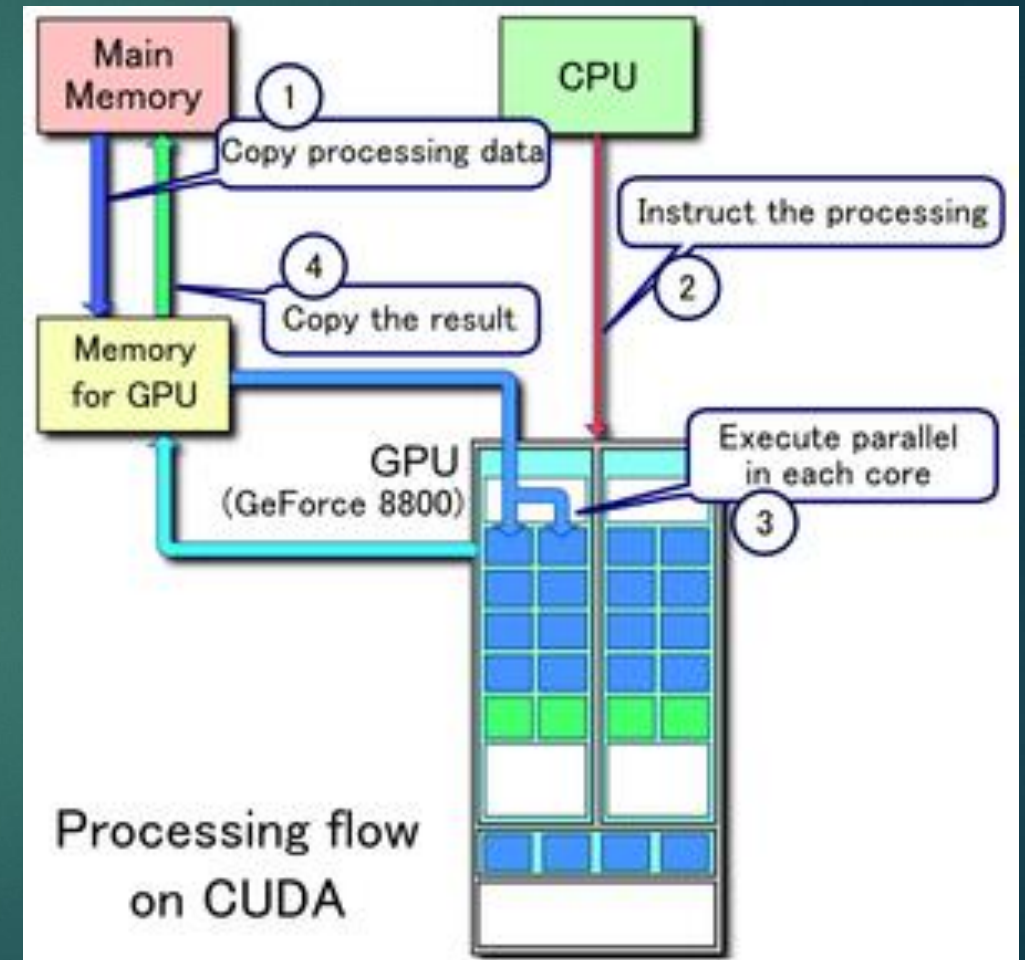
14



General Program Structure

15

- ▶ Initialize Device
- ▶ Create Device context
- ▶ Build Device program (Kernel)
- ▶ Allocate Device memory
- ▶ Transfer data from Host to Device
- ▶ Invoke Device functions
- ▶ Transfer data from Device to Host
- ▶ Deallocate Device memory
- ▶ Finalize Device usage



Create Context

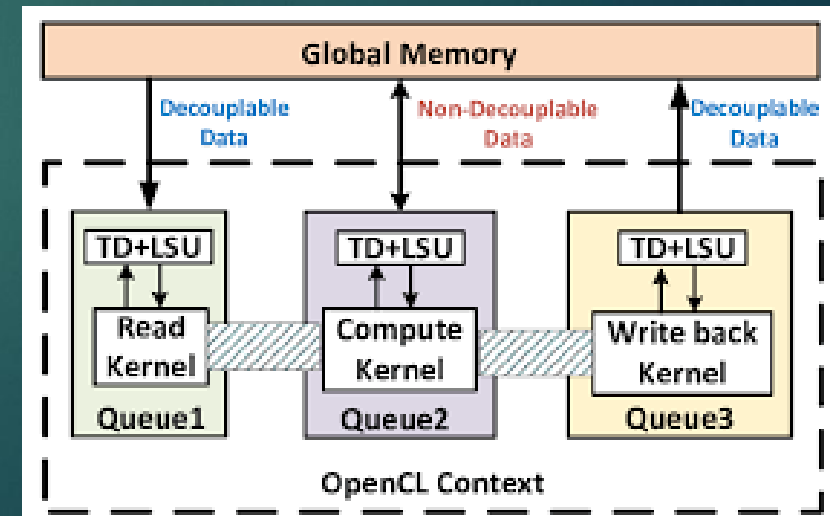
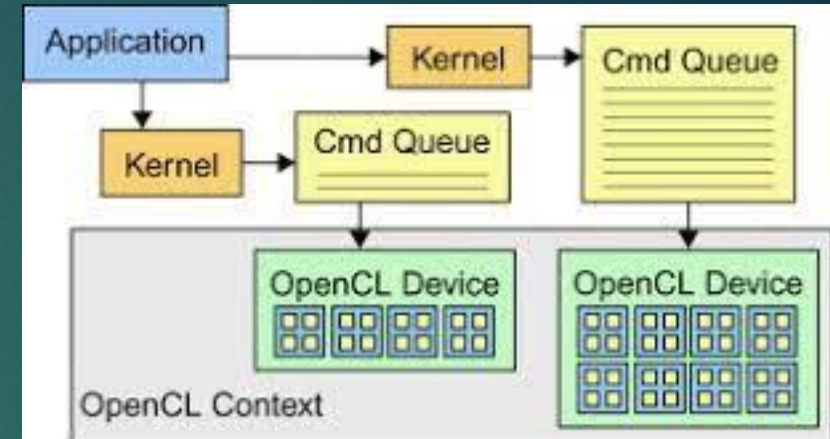
16

► CUDA

- **Context creation is implicit** in single-threaded programs
- Multiple contexts can be explicitly created
- Each **thread maintains a context stack**
 - Top context is current context
- Threads
 - Contexts can be swapped between threads
 - A thread can only have one context active at a time (stack)
 - **A context cannot be shared simultaneously between threads**

► OpenCL

- All commands explicitly associated with a context
- Must **create a command queue to invoke**



Initialize Device

17

► CUDA

- `cudaGetDeviceCount()`
- `cudaSetDevice()`
- `cudaGetDeviceProperties()`

```
CUDA::CUDA(int Device) : Base()
{
    mValid = false;
    int DeviceCount;
    cudaGetDeviceCount( &DeviceCount );
    if (!DeviceCount)
    {
        return;
    }

    Device = Device == -1 ? DeviceCount - 1 : Device;
    cudaSetDevice( Device );
    mValid = true;
}
```

Initialize Device

► OpenCL

- **Context must be built before anything can be done** on the GPU/Device
- All commands are with respect to a given context

```
OpenCL::OpenCL(int Device) : Base()
{
    init();                // Initialize class pointers to NULL
    cl_int RC;
    mGPUContext = clCreateContextFromType( 0, CL_DEVICE_TYPE_GPU, NULL, NULL, &RC );
    size_t Bytes;
    RC = clGetContextInfo( mGPUContext, CL_CONTEXT_DEVICES, 0, NULL, &Bytes );
    int NumDevices = Bytes / sizeof( cl_device_id );

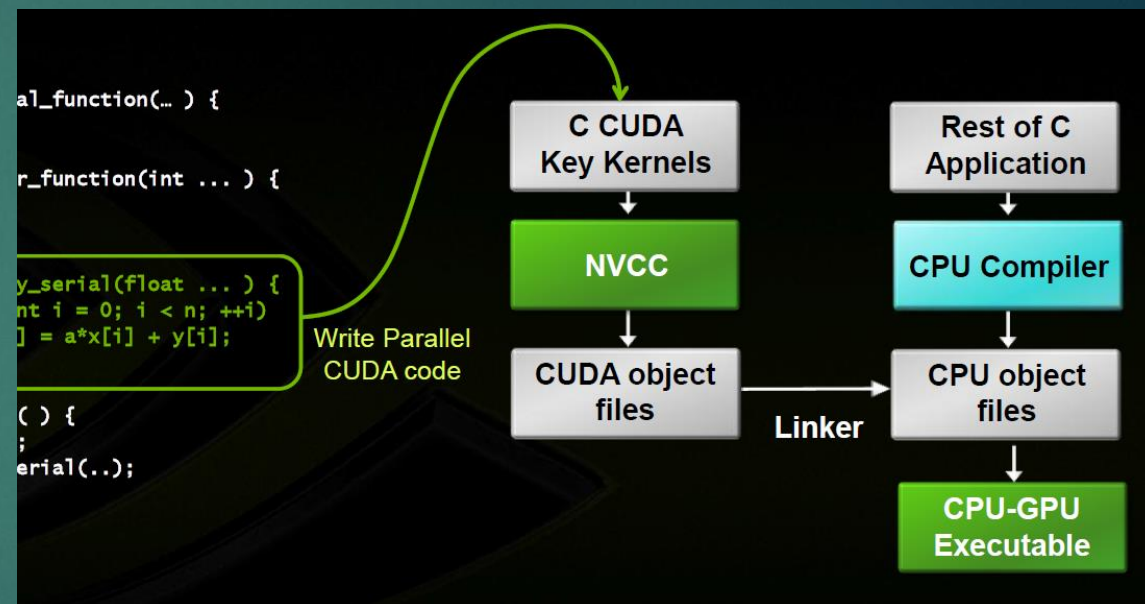
    cl_device_id *Devices = new cl_device_id[ NumDevices ];
    RC = clGetContextInfo( mGPUContext, CL_CONTEXT_DEVICES, Bytes, Devices, NULL );

    mCommandQueue = clCreateCommandQueue( mGPUContext, Devices[ Device ], 0, &RC );
    size_t MaxWorkItemSizes[ 256 ];
    RC = clGetDeviceInfo( Devices[ Device ], CL_DEVICE_MAX_WORK_ITEM_SIZES,
                        sizeof( MaxWorkItemSizes ), MaxWorkItemSizes, NULL );
    mMaxWorkItems = MaxWorkItemSizes[ 0 ];
    mMaxWorkItemsMask = ~(mMaxWorkItems - 1);
}
```

Build Device Program

► CUDA

- GPU code is **compiled using nvcc compiler**
- Object code is statically bound to CPU executable
 - GPU code is intrinsically part of the program
 - **Mapping of problem to threads performed at compile time**

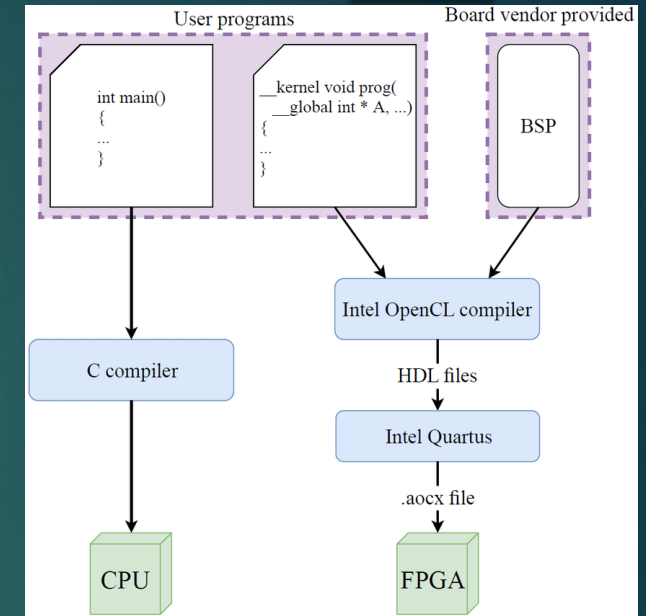
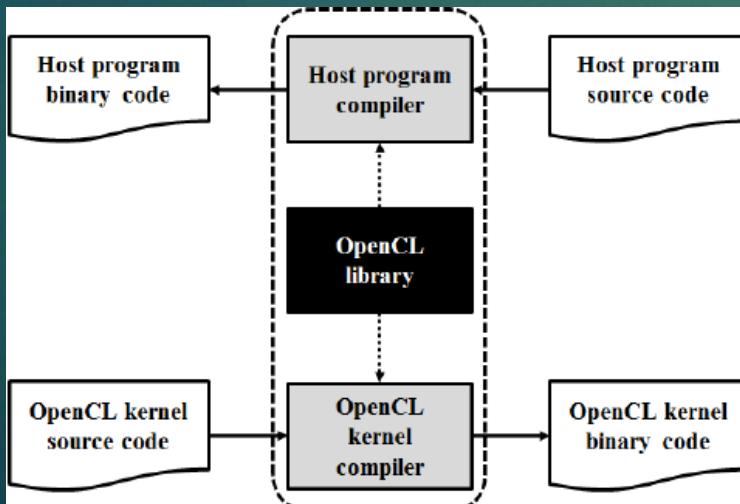


Build Device Program

20

► OpenCL

- GPU code is bound at runtime to the GPU
- OpenCL compiler is part of executable
- Code can be source code or object code
 - Source code can be dynamically generated by the program
 - Can be stored in an external file



// Continued from constructor

```
char *code = shrFindFilePath( "code.cl", "." );
size_t CodeLength = 0;
char *Source = oclLoadProgSource( myCode, "", &CodeLength );
const char *SourceCode = Source;
mProgram = clCreateProgramWithSource( mGPUContext, 1, &SourceCode,
                                     &CodeLength, &RC );

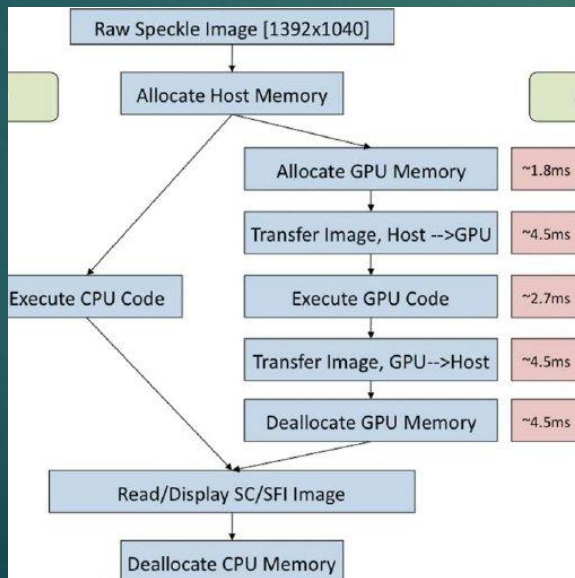
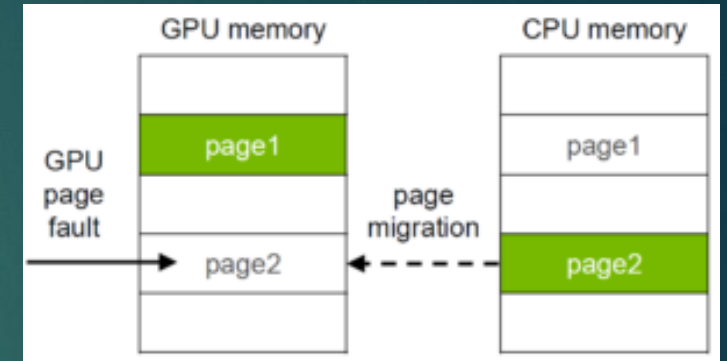
RC = clBuildProgram( mProgram, 0, NULL, NULL, NULL, NULL );
std::free( code );
std::free( Source );
mValid = RC == CL_SUCCESS;
}
```


Allocate/Deallocate Device Memory

21

► CUDA

- Most frequently used allocator: **cudaMalloc()**
- **Returns a memory pointer to GPU memory**
- Memory pointer **cannot be used by CPU directly**
 - Passed to GPU calls



```
void *CUDA::malloc(size_t Bytes)
{
    void *Memory;
    cudaError_t RC = cudaMalloc( &Memory, Bytes );
    return( RC == cudaSuccess ? Memory : NULL );
}

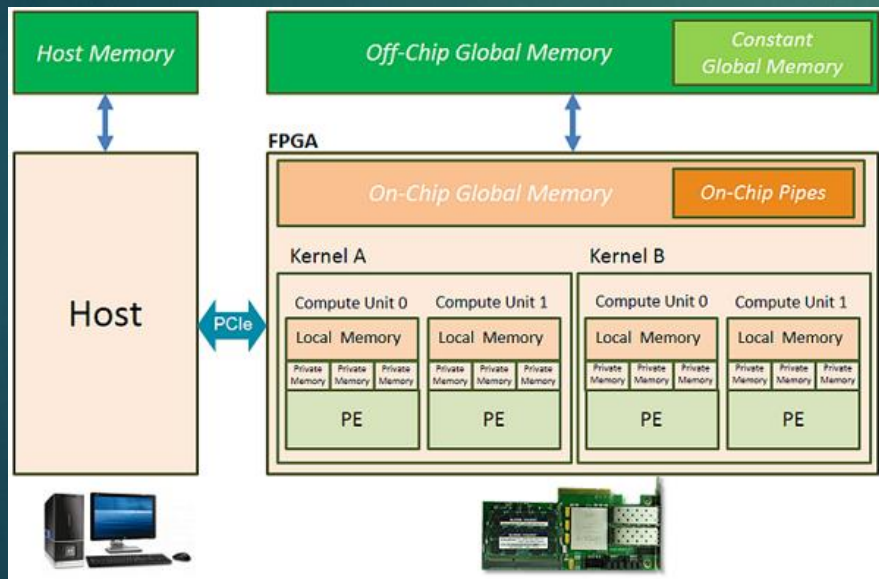
void CUDA::free(void *Memory)
{
    if (Memory)
    {
        cudaFree( Memory );
    }
}
```

Allocate/Deallocate Device Memory

22

► OpenCL

- Like all things, **memory allocation explicitly performed within a context**



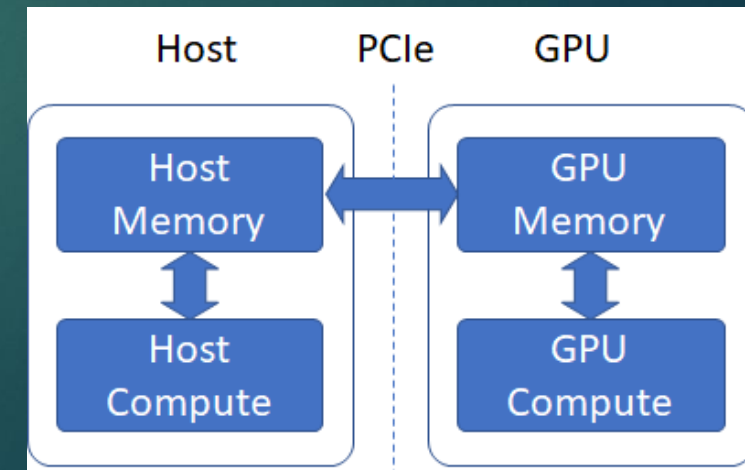
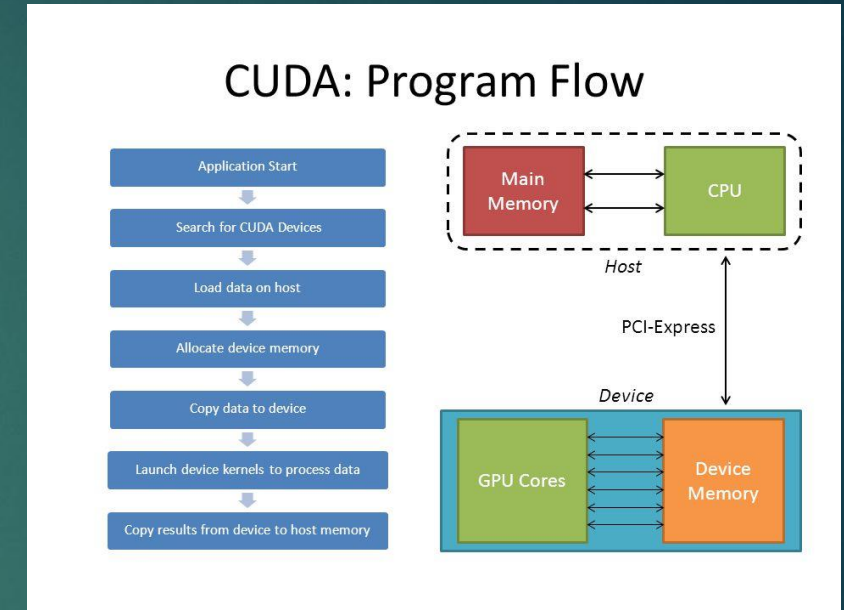
```
void *OpenCL::malloc(size_t NumBytes)
{
    size_t Size = NumBytes / 32 + (NumBytes & 31 ? 1 : 0);
    cl_int RC;
    cl_mem Memory = clCreateBuffer( mGPUContext, CL_MEM_READ_WRITE,
                                   Size, NULL, &RC );
    return( RC == CL_SUCCESS ? Memory : NULL );
}
```

```
void OpenCL::free(void *Memory)
{
    if (Memory)
    {
        cl_mem *Ptr = reinterpret_cast<cl_mem>( Memory );
        clReleaseMemObject( Memory );
    }
}
```

Host/Device Data Transfer

23

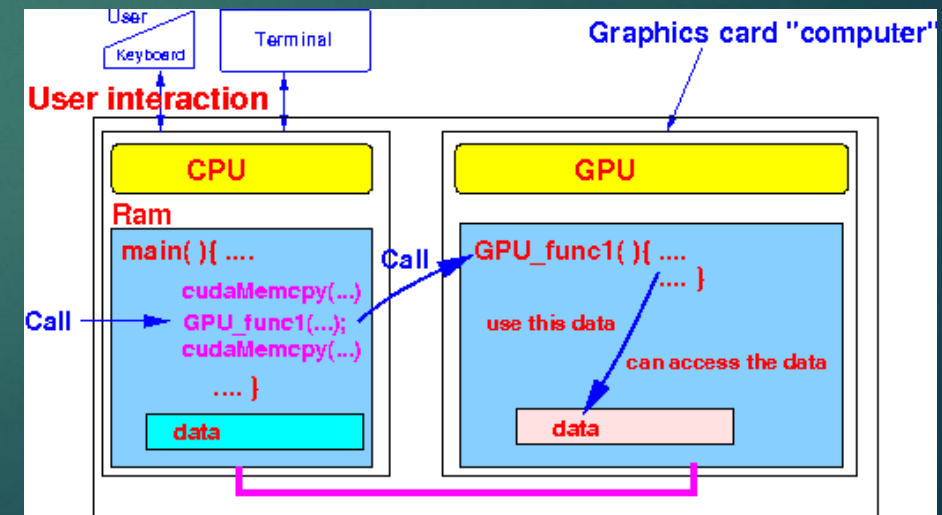
- ▶ **Data moved across PCIe bus**
- ▶ CUDA
 - ▶ Data transfer accomplished via **cudaMemcpy()** routine
 - ▶ Implicit synchronization point
 - ▶ Non-blocking copies are available
 - ▶ Direction is determined by enumeration
 - ▶ cudaMemcpyHostToDevice
 - ▶ cudaMemcpyDeviceToHost
 - ▶ Allocated memory can be bound to texture memory
 - ▶ cudaBindTexture
- ▶ OpenCL
 - ▶ Memory transfer via clEnqueueWriteBuffer() and clEnqueueReadBuffer()
 - ▶ **Synchronization controlled by parameters to calls**
 - ▶ Default is non-blocking



Call Device Functions (Kernels)

24

- ▶ Functions in CPU are **executed when invoked**
- ▶ Device function calls from CPU create **execution queue**
 - ▶ CPU does not wait until Device function completes – command is simply queued
 - ▶ Device executes commands on the queue **using its own ordering**
 - ▶ Synchronization points cause CPU to stall to wait for Device return
- ▶ CUDA
 - ▶ **cudaThreadSynchronize()**



Device Function Calls

25

- ▶ Device function calls have an **associated dimensionality (which can be 1D, 2D or 3D)**

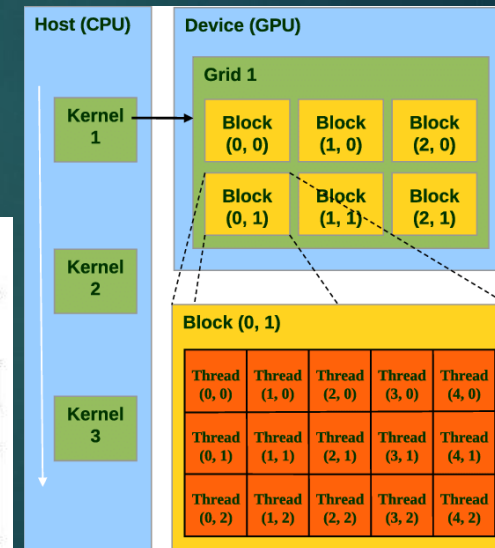
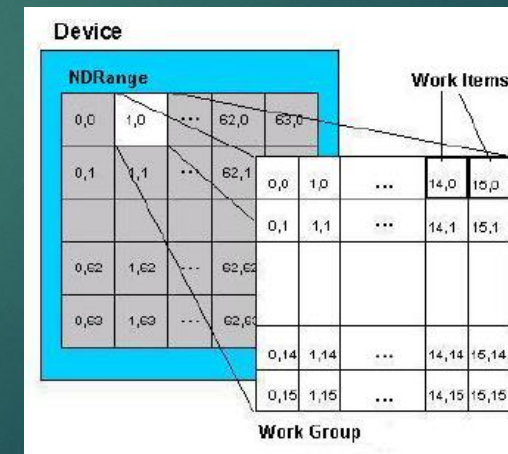
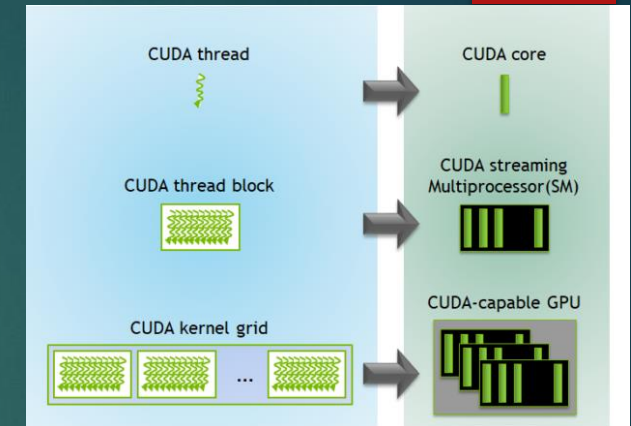
- ▶ CUDA

- ▶ Extended language syntax to include problem dimension
 - ▶ Syntax

```
function<<<dimBlock,dimGrid>>>( arguments );
```

- ▶ OpenCL

- ▶ **Must explicitly put function arguments into context**
clSetKernelArg()
 - ▶ Invoke kernel using the context
 - ▶ Kernel retrieves arguments from context automatically



Device Cleanup/Termination

26

▶ CUDA

- ▶ **Manages most cleanup operations automatically** as a context is destroyed

▶ OpenCL

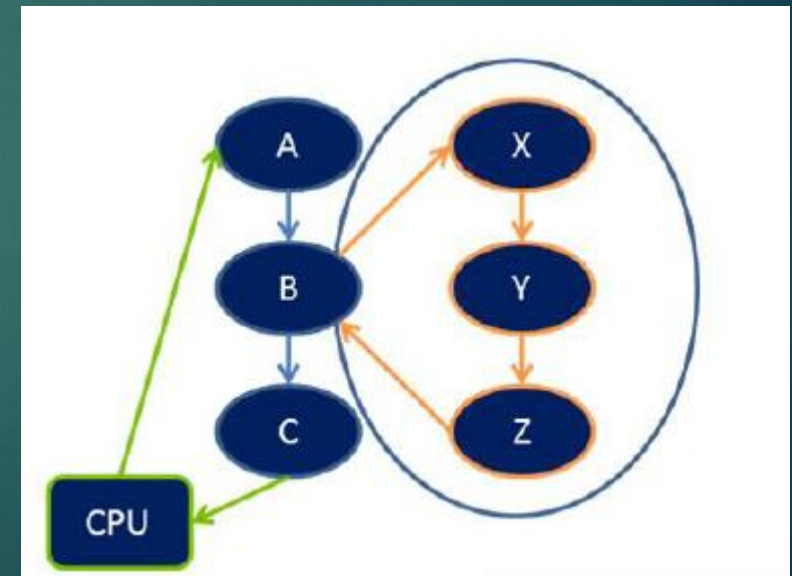
- ▶ **Provides low-level APIs for deallocation** of all resources
- ▶ Invoked in order opposite to invocation

`clReleaseKernel()`

`clReleaseProgram()`

`clReleaseCommandQueue()`

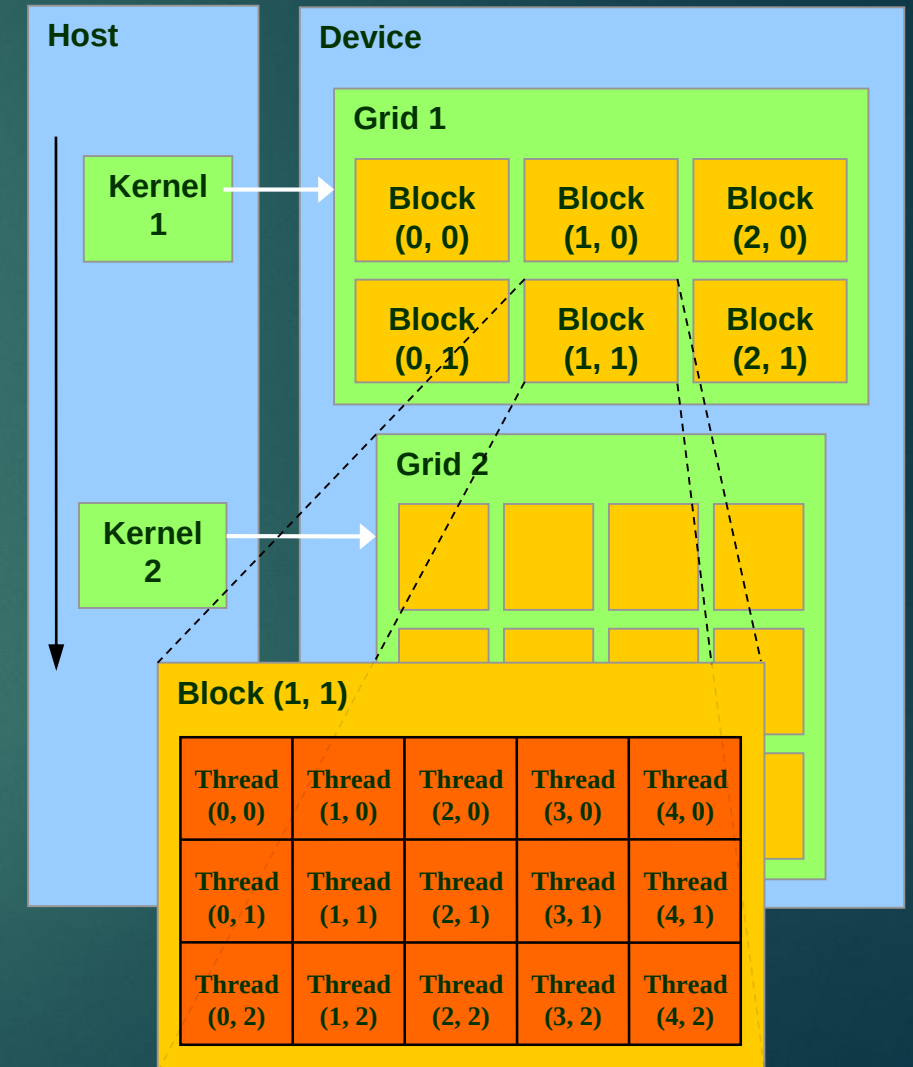
`clReleaseContext()`



Thread Batching: Grids and Blocks

27

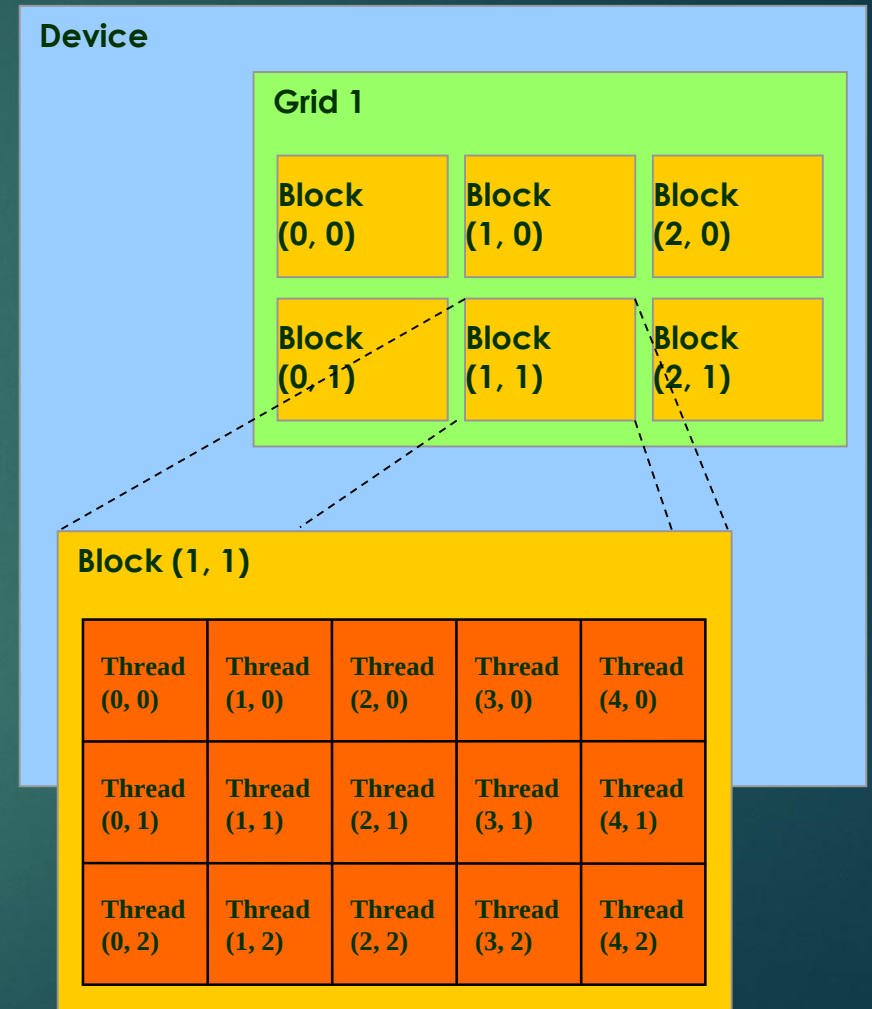
- ▶ A kernel is executed as a **grid of thread blocks (aka blocks)**
- ▶ A **thread block** is a batch of threads that can **cooperate** with each other by:
 - ▶ Synchronizing their execution
 - ▶ Diverging execution results in performance loss
 - ▶ Efficiently sharing data through a low latency **shared memory**
- ▶ Two threads from two different blocks cannot cooperate



Block and Thread IDs

28

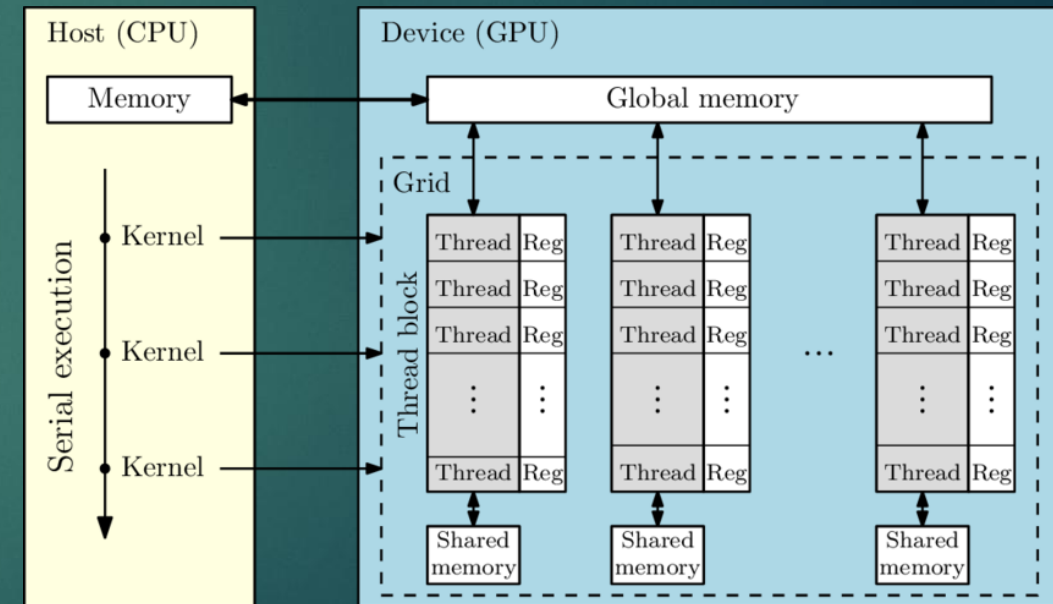
- ▶ Threads and blocks have IDs
 - ▶ So each thread can identify what data they will operate on
 - ▶ Block ID: 1D or 2D
 - ▶ Thread ID: 1D, 2D, or 3D
- ▶ **Simplifies memory addressing** when processing multidimensional data
 - ▶ Image processing
 - ▶ Solving PDEs on volumes
 - ▶ Other problems with underlying 1D, 2D or 3D geometry



Device Kernels

29

- ▶ **Each function is passed data to create a unique ID**
 - ▶ Data typically specifies “spatial coordinates” of function execution processor within the hardware
 - ▶ The ID is used to coordinate data access
 - ▶ **Ensures that two threads’ accesses do not collide**
- ▶ CUDA function types
 - ▶ **__global__**
 - ▶ Callable by CPU
 - ▶ Cannot be called by GPU/Device
 - ▶ **__device__**
 - ▶ Callable by other GPU/Device functions
 - ▶ Cannot be called by CPU
 - ▶ CUDA expands these as inline functions via nvcc
 - ▶ Adds to function resource utilization



OpenCL Kernel Invocation

30

- Use C++ templates to simplify argument handling

```
template<typename T> inline cl_int setArg(cl_kernel Kernel, unsigned Pos, T Arg)
{
    return( clSetKernelArg( Kernel, Pos, sizeof( T ), &Arg ) );
}
template<> inline cl_int setArg(cl_kernel Kernel, unsigned Pos, size_t SharedSize)
{
    // This routine, unlike the others, sets up shared memory by passing
    // NULL in as the pointer to the variable.
    return( clSetKernelArg( Kernel, Pos, SharedSize, NULL ) );
}
template<> inline cl_int setArg(cl_kernel Kernel, unsigned Pos, int Arg)
{
    cl_int ArgInt = Arg;
    return( clSetKernelArg( Kernel, Pos, sizeof( ArgInt ), &ArgInt ) );
}
template<> inline cl_int setArg(cl_kernel Kernel, unsigned Pos, float Arg)
{
    cl_float ArgFloat = Arg;
    return( clSetKernelArg( Kernel, Pos, sizeof( ArgFloat ), &ArgFloat ) );
}
...
template<typename T0> inline cl_int setArgs(cl_kernel Kernel, T0 Arg0)
{
    return( setArg( Kernel, 0, Arg0 ) );
}
template<typename T0, typename T1> inline cl_int setArgs(cl_kernel Kernel, T0 Arg0, T1 Arg1)
{
    return( setArg( Kernel, 0, Arg0 ) | setArg( Kernel, 1, Arg1 ) );
}
template<typename T0, typename T1, typename T2> inline cl_int setArgs(cl_kernel Kernel, T0 Arg0, T1 Arg1, T2 Arg2)
{
    return( setArg( Kernel, 0, Arg0 ) | setArg( Kernel, 1, Arg1 ) | setArg( Kernel, 2, Arg2 ) );
}
...
```

OpenCL Kernel Invocation

31

- ▶ BLAS-like example (Basic Linear Algebra Subprograms)
 - ▶ **CUDA provides BLAS library; OpenCL doesn't**
 - ▶ Must write own BLAS routines in OpenCL to port between the two easily
 - ▶ swap() function swaps contents of 2 vectors with differing vector strides

```
void OpenCL::blasSswap(int n, float *x, int incx, float *y, int incy)
{
    if (!checkBLASKernel( &mSswapKernel, "Sswap" ))
    {
        return;
    }
    mLastBLASStatus = Base::BLAS_INTERNAL_ERROR;
    if (x && y)
    {
        if (setArgs( mSswapKernel, n, x, incx, y, incy ) == CL_SUCCESS)
        {
            executeBLASKernel( mSswapKernel, n );
        }
    }
}
```

OpenCL Kernel Invocation

32

► BLAS support functions

```
bool OpenCL::checkBLASKernel(cl_kernel *Kernel, const char *KernelName)
{
    if (!mValid)
    {
        mLastBLASStatus = Base::BLAS_NOT_INITIALIZED;
        return( false );
    }
    if (!(*Kernel))
    {
        cl_int RC;
        *Kernel = clCreateKernel( mProgram, KernelName, &RC );
        if (RC != CL_SUCCESS)
        {
            mLastBLASStatus = Base::BLAS_INTERNAL_ERROR;
            return( false );
        }
    }
    return( true );
}

inline void OpenCL::executeBLASKernel(cl_kernel Kernel, int n)
{
    size_t Size = n;
    size_t GlobalWorkSize = Size & mMaxWorkItemsMask;
    if (Size & ~mMaxWorkItemsMask)
    {
        GlobalWorkSize += mMaxWorkItems;
    }
    cl_int RC = clEnqueueNDRangeKernel( mCommandQueue, Kernel, 1, NULL, &GlobalWorkSize,
                                         &mMaxWorkItems, 0, NULL, NULL );

    clFinish( mCommandQueue );
    mLastBLASStatus = (RC == CL_SUCCESS) ? Base::BLAS_SUCCESS : Base::BLAS_EXECUTION_FAILED;
}
```


OpenCL Kernels

33

► BLAS SSWAP example

```
__kernel void Sswap(__global int n, __global float *x,
                    __global int incx, __global float *y,
                    __global int incy)
{
    const unsigned GID = get_global_id( 0 );
    if (GID < n)
    {
        int lx = (incx >= 0) ? 0 : ((1 - n) * incx);
        int ly = (incy >= 0) ? 0 : ((1 - n) * incy);
        float temp = y[ ly + GID * incy ];
        y[ ly + GID * incy ] = x[ lx + GID * incx ];
        x[ lx + GID * incx ] = temp;
    }
}
```

CUDA Kernels

34

► CPU

```
#include "kernel.cu"
...
{
    const unsigned int size_x = 256;
    const unsigned int size_y = 4096;
    ...
    dim3 grid(size_x / BLOCK_DIM, size_y / BLOCK_DIM, 1);
    dim3 threads(BLOCK_DIM, BLOCK_DIM, 1);
    transpose_naive<<< grid, threads >>>(d_odata, d_idata, size_x, size_y);
    cudaThreadSynchronize();
    ...
}
```

► GPU (kernel.cu)

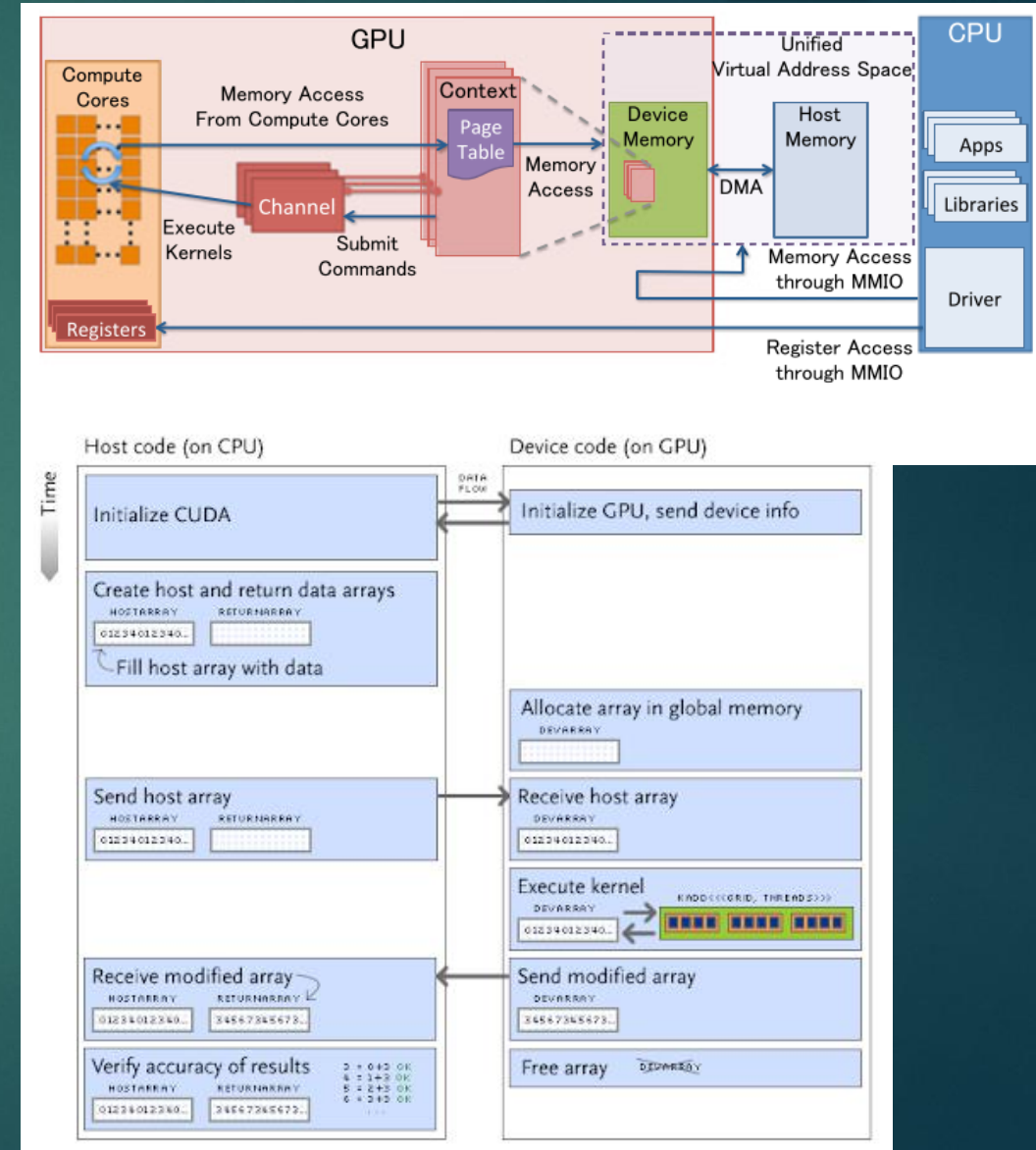
```
#define BLOCK_DIM 16

__global__ void transpose_naive(float *odata, float* idata, int width, int height)
{
    unsigned int xIndex = blockDim.x * blockIdx.x + threadIdx.x;
    unsigned int yIndex = blockDim.y * blockIdx.y + threadIdx.y;
    if (xIndex < width && yIndex < height)
    {
        unsigned int index_in  = xIndex + width * yIndex;
        unsigned int index_out = yIndex + height * xIndex;
        odata[index_out] = idata[index_in];
    }
}
```

Conclusion

35

- ▶ Computing Programming Steps
- ▶ Host and Devices
 - ▶ Compute and Memory
- ▶ Initial Devices
- ▶ Transfer Codes and Data to Devices
- ▶ Start and Finish Kernel Execution
- ▶ Transfer Results to Host
- ▶ Terminate and Cleanup
- ▶ CUDA vs OpenCL



END

QUESTIONS & ANSWERS